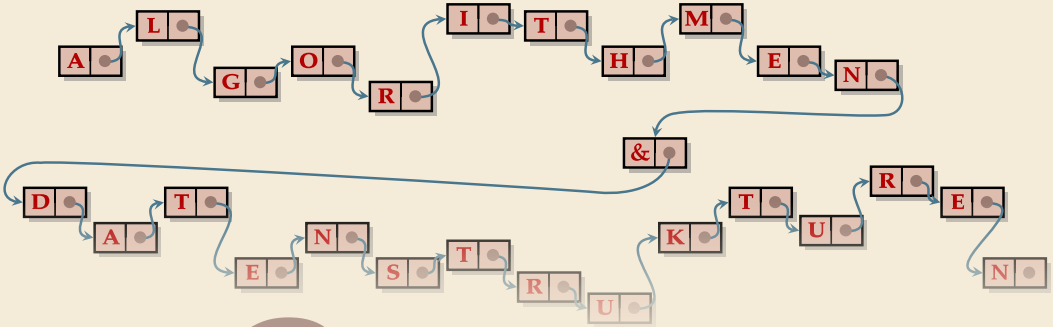


3 Mathematische Grundlagen

Algorithmen & Datenstrukturen · Sommersemester 2026

Prof. Dr. Sebastian Wild

CS210 (Sommersemester 2026)
Philipps-Universität Marburg
version `2026-04-29 10:07`

Outline

3 Mathematische Grundlagen

3.1 Beispiel: 3SUM
3.2 Folgen & Grenzwerte
3.3 Asymptotische Notation
3.4 Rechenregeln für Asymptotiken
3.5 Rekursiv definierte Folgen
3.6 Das Master-Theorem
3.7 Grundlagen Stochastik

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

→

→

→

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

2m

$\xrightarrow{\sim 10x}$

12m

$\xrightarrow{\sim 10x}$

170m

$\xrightarrow{\sim 10x}$

1280m

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

2m	$\xrightarrow{\sim 10x}$	12m	$\xrightarrow{\sim 10x}$	170m	$\xrightarrow{\sim 10x}$	1280m
€ 50 000	$\xrightarrow{\sim 10x}$	€ 350 000	$\xrightarrow{\sim 50x}$	€ 22 000 000	$\xrightarrow{\sim 200x}$	€ 4 500 000 000

(Kosten sind Schätzungen der heutigen Kosten)

Recap: Skalierbarkeit

Analogie: Architektur

- 1m Brücke: Holzplanke genügt!
- 10x $\rightsquigarrow$ Konstruktion nötig
- 1000x $\rightsquigarrow$ Meisterleistung der Bauingenieurskunst

Informatik

- Eingabegrößen in Informatiksystemen erreichen oft Faktor 1 000 000 000!

  1 Byte $\longrightarrow$ 1 GB

$\rightsquigarrow$ *Können uns keine Holzbretter leisten.*

Eine 1km Brücke ist nicht einfach ein 1000-fach größeres Brett.

$\longrightarrow$

$\longrightarrow$

$\longrightarrow$

2m	$\xrightarrow{\sim 10x}$	12m	$\xrightarrow{\sim 10x}$	170m	$\xrightarrow{\sim 10x}$	1280m
€ 50 000	$\xrightarrow{\sim 10x}$	€ 350 000	$\xrightarrow{\sim 50x}$	€ 22 000 000	$\xrightarrow{\sim 200x}$	€ 4 500 000 000

(Kosten sind Schätzungen der heutigen Kosten)

Wie quantifizieren wir “Skalierbarkeit”?

- ⇝ *Für Algorithmen (in erster Näherung)* ***Wachstum*** *der Kosten in Eingabegröße entscheidend.*
- Tricks in der Implementierung und bessere Hardware retten uns nicht
- tragen aber natürlich zu den Gesamtkosten bei
- ⇝ für Performance-Bottlenecks durch Experten noch etwas rauszuholen
- ⇝ den deutlich größeren Impact hat aber die Wahl der richtigen Algorithmen und Datenstrukturen

Wie quantifizieren wir "Skalierbarkeit"?

⇝ *Für Algorithmen (in erster Näherung)* ***Wachstum*** *der Kosten in Eingabegröße entscheidend.*

- Tricks in der Implementierung und bessere Hardware retten uns nicht
- tragen aber natürlich zu den Gesamtkosten bei

⇝ für Performance-Bottlenecks durch Experten noch etwas rauszuholen

⇝ den deutlich größeren Impact hat aber die Wahl der richtigen Algorithmen und Datenstrukturen

Wie können wir also Algorithmen bezüglich des ***Wachstums*** *der Kosten* ***vergleichen****?*

- Sprache für Performancecharakteristika: **Asymptotische Approximationen**
  O-Notation (plus Varianten und Verfeinerungen)

⇝ Damit müssen Sie souverän umgehen lernen.

Wie quantifizieren wir "Skalierbarkeit"?

⇝ *Für Algorithmen (in erster Näherung)* ***Wachstum*** *der Kosten in Eingabegröße entscheidend.*

- Tricks in der Implementierung und bessere Hardware retten uns nicht
- tragen aber natürlich zu den Gesamtkosten bei

⇝ für Performance-Bottlenecks durch Experten noch etwas rauszuholen

⇝ den deutlich größeren Impact hat aber die Wahl der richtigen Algorithmen und Datenstrukturen

Wie können wir also Algorithmen bezüglich des ***Wachstums*** *der Kosten* ***vergleichen****?*

- Sprache für Performancecharakteristika: **Asymptotische Approximationen**
  O-Notation (plus Varianten und Verfeinerungen)

⇝ Damit müssen Sie souverän umgehen lernen.

Dazu legen wir hier die Grundlagen und geben die formalen Definitionen.

3.1 Beispiel: 3SUM

3SUM

3SUM-Problem

- **Gegeben:** $A[0..n)$ with $A[i] \in \mathbb{Z}$ for $0 \leq i < n$.
- **Ziel:** Anzahl c von Tripeln aus A mit Summe 0

$$c \quad = \quad \left|\left\{(i,j,k) \;:\; 0 \leq i < j < k < n \;\wedge\; A[i] + A[j] + A[k] = 0\right\}\right|$$

Parameter:

- Eingabegröße: # Zahlen n

3SUM

3SUM-Problem

- **Gegeben:** $A[0..n)$ with $A[i] \in \mathbb{Z}$ for $0 \leq i < n$.
- **Ziel:** Anzahl c von Tripeln aus A mit Summe 0

$$c \;=\; \left|\left\{(i,j,k) \;:\; 0 \leq i < j < k < n \;\wedge\; A[i] + A[j] + A[k] = 0\right\}\right|$$

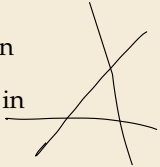
Parameter:

- Eingabegröße: #Zahlen n

Anwendungen:

3 Punkte auf einer Geraden

- kollineare Tripel von Punkten finden
- Tripel von Geraden finden, die sich in einem Punkt schneiden



- Dreiecke aus Punkten formen mit kleinem Flächeninhalt
- ...

Clicker Question

$[a, b) = \{x \in \mathbb{R} : a \le x < b\}$

$\{a, a+1, \ldots, b-1\}$
$\parallel$
$[a..b) = \{n \in \mathbb{Z} : a \le n < b\}$

Was ist die Ausgabe für 3SUM mit folgender Eingabe?

$A[0..6) = [1, 0, -1, 3, 2, -1]$

0 1 2 3 4 5

(0,1,2)
(0,1,5)
(2,4,5)

$A[i]$

A `ArrayIndexOutOfBoundsException`

B 0

C 1

D 2

E 3

F 4

G 4.5

H 5

I 6

→ sli.do/cs210

Clicker Question

Was ist die Ausgabe für 3SUM mit folgender Eingabe?

$A[0..6) = [1, 0, -1, 3, 2, -1]$

- **A** ~~`ArrayIndexOutOfBoundsException`~~
- **B** ~~0~~
- **C** ~~1~~
- **D** ~~2~~
- **E** 3 ✓
- **F** ~~4~~
- **G** ~~4.5~~
- **H** ~~5~~
- **I** ~~6~~

→ sli.do/cs210

Einschub: Template für Entwurf von Algorithmen

1. **Algorithmische Idee**
   Kurze abstrakte Idee, die dem Algorithmus zu Grunde liegt (Freitext)
   (ein Experte könnte den Rest von hier ergänzen)

2. **Pseudocode**
   struktierte Beschreibung der Methode inklusive Rand- und Basisfälle
   eindeutig, präzise; nah an echtem Code

3. **Korrektheitsbeweis**
   Argument, warum der Algorithmus das korrekte Ergebnis liefert
   oft per Induktion unter Verwendung von Invarianten

4. **Analyse**
   Analyse der Kosten des Algorithmus
   i. d. R. Θ-Klasse für den Worst Case;
   wo interessant auch Speicherbedarf

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

2. **Pseudocode**

```
c = 0
for i = 0, ..., n − 3
    for j = i + 1, ..., n − 2
        for k = j + 1, ..., n − 1
            t = A[i] + A[j] + A[k]
            if t == 0 then c := c + 1
        end for
    end for
end for
return c
```

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

2. **Pseudocode**

```
c = 0
for i = 0, ..., n − 3
    for j = i + 1, ..., n − 2
        for k = j + 1, ..., n − 1
            t = A[i] + A[j] + A[k]
            if t == 0 then c := c + 1
        end for
    end for
end for
return c
```

3. **Korrektheitsbeweis**
   direkt nach Definition von c

$$c = \left|\left\{(i,j,k) : 0 \le i < j < k < n \wedge A[i] + A[j] + A[k] = 0\right\}\right|$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

2. **Pseudocode**

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. **Korrektheitsbeweis**
   direkt nach Definition von c

Analysen später nochmal im Detail

4. **Analyse**

   # Ausführungen von Zeilen 5–6

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. **Algorithmische Idee**
   Alle Tripel durchprobieren

2. **Pseudocode**

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. **Korrektheitsbeweis**
   direkt nach Definition von c

Analysen später nochmal im Detail

4. **Analyse**

   # Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
c = 0
for i = 0, ..., n − 3
    for j = i + 1, ..., n − 2
        for k = j + 1, ..., n − 1
            t = A[i] + A[j] + A[k]
            if t == 0 then c := c + 1
        end for
    end for
end for
return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$|[a..b]| = b - a + 1$

$n - (i+1+1)$

$= n - i - 2$

$n - (n-2+1)$

$= 1$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2},$$

$1 + 2 + 3 + 4$
$+ 4 + 3 + 2 + 1$
$5 + 5 \cdot \cdot 5$

https://www.wild-inter.net/tcscs.pdf

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$n-3$ $\quad$ $n-1$

$$\frac{1 \cdot 2}{2} \qquad \frac{(n-2)(n-1)}{2}$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$$= \frac{1}{2} \sum_{i=1}^{n-2} i(i+1) = \frac{1}{2} \sum_{i=1}^{n-2} i^2 + \frac{1}{2} \sum_{i=1}^{n-2} i$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2}, \quad \sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6},$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$$= \frac{1}{2} \sum_{i=1}^{n-2} i(i+1) = \frac{1}{2} \sum_{i=1}^{n-2} i^2 + \frac{1}{2} \sum_{i=1}^{n-2} i$$

$$= \frac{(n-2)(n-1)(2(n-2)+1)}{12} + \frac{(n-2)(n-1)}{4}$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$$= \frac{1}{2} \sum_{i=1}^{n-2} i(i+1) = \frac{1}{2} \sum_{i=1}^{n-2} i^2 + \frac{1}{2} \sum_{i=1}^{n-2} i$$

$$= \frac{(n-2)(n-1)(2(n-2)+1)}{12} + \frac{(n-2)(n-1)}{4}$$

$$= \frac{(n-2)(n-1)n}{6} = \binom{n}{3}$$

Brute-force 3SUM

- allgemeiner Ratschlag: erstmal die denkbar einfachste Lösung versuchen

1. Algorithmische Idee
Alle Tripel durchprobieren

2. Pseudocode

```
1  c = 0
2  for i = 0, ..., n − 3
3      for j = i + 1, ..., n − 2
4          for k = j + 1, ..., n − 1
5              t = A[i] + A[j] + A[k]
6              if t == 0 then c := c + 1
7          end for
8      end for
9  end for
10 return c
```

3. Korrektheitsbeweis
direkt nach Definition von c

Analysen später nochmal im Detail

4. Analyse

Ausführungen von Zeilen 5–6

$$= \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} \sum_{k=j+1}^{n-1} 1 = \sum_{i=0}^{n-3} \sum_{j=i+1}^{n-2} (n-(j+1))$$

$$= \sum_{i=0}^{n-3} \sum_{j=1}^{n-i-2} j = \sum_{i=0}^{n-3} \frac{(n-i-2)(n-i-1)}{2}$$

$$= \frac{1}{2} \sum_{i=1}^{n-2} i(i+1) = \frac{1}{2} \sum_{i=1}^{n-2} i^2 + \frac{1}{2} \sum_{i=1}^{n-2} i$$

$$= \frac{(n-2)(n-1)(2(n-2)+1)}{12} + \frac{(n-2)(n-1)}{4}$$

$$= \frac{(n-2)(n-1)n}{6}$$

Ist das jetzt gut??

Einschub: Überschlagsrechnungen (*back-of-the-envelope math*)

Ob $T(n) = \frac{(n-2)(n-1)n}{6}$ Operationen machbar sind, hängt von n ab!

Ganz grobe Überschlagsrechnungen sind hier hilfreich!

n	$T(n)$	**Zeit bei 1GHz / 1ns pro Operation**
10	120	instant
100	161 700	0.1 ms
1 000	166 167 000	0.16 s
10 000	166 616 670 000	2:40 min
100 000	166 661 666 700 000	46 h (!)

$\rightsquigarrow$ Arrays mit 10 000 Zahlen grade noch machbar, 100 000 eher nicht.

Einschub: Überschlagsrechnungen (*back-of-the-envelope math*)

Ob $T(n) = \dfrac{(n-2)(n-1)n}{6}$ Operationen machbar sind, hängt von n ab!

Ganz grobe Überschlagsrechnungen sind hier hilfreich!

n	$T(n)$	**Zeit bei 1GHz / 1ns pro Operation**
10	120	instant
100	161 700	0.1 ms
1 000	166 167 000	0.16 s
10 000	166 616 670 000	2:40 min
100 000	166 661 666 700 000	46 h (!)

⇝ Arrays mit 10 000 Zahlen grade noch machbar, 100 000 eher nicht.

Übrigens: Man kann 3SUM deutlich schneller lösen . . .
werden wir noch einmal wiedersehen 😉

3.2 Folgen & Grenzwerte

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen

$\mathbb{Z}$

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen
- $\lg = \log_2$; $\ln = \log_e$ $e \approx 2.71$ $\cancel{\log_0}$

  ld

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen
- $\lg = \log_2$; $\ln = \log_e$

Folgen

- (Reelle) *Zahlenfolge* $(a_n)_{n \in \mathbb{N}_{\geq 1}}$
  formal eine Funktion $a : \mathbb{N}_{\geq 1} \to \mathbb{R}$
  schreiben auch "$a_1, a_2, a_3, \ldots$"

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen
- $\lg = \log_2$; $\ln = \log_e$

Folgen

"$a_n = n^2$" $(a : n \mapsto n^2)$

- (Reelle) *Zahlenfolge* $(a_n)_{n \in \mathbb{N}_{\geq 1}}$
  formal eine Funktion $a : \mathbb{N}_{\geq 1} \to \mathbb{R}$
  schreiben auch "$a_1, a_2, a_3, \ldots$"
- *Grenzwert* einer Folge
  $\lim\limits_{n \to \infty} a_n = a^*$ genau dann, wenn $\boxed{\overset{\text{für alle}}{\forall \varepsilon > 0} \ \overset{\text{existiert}}{\exists n_0 \in \mathbb{N}_{\geq 1}} \ \forall n \geq n_0 : |a_n - a^*| < \varepsilon}$

Definitionen zur Erinnerung

Ein bisschen Notation

- $\mathbb{N}_0 = \{0, 1, 2, 3, \ldots, \}$ natürliche Zahlen mit 0
- $\mathbb{N}_{\geq 1} = \{1, 2, 3, \ldots, \}$ natürliche Zahlen ab 1
- $\cancel{\mathbb{N}}$: (Uneindeutigkeit vermeiden)
- $\mathbb{R}$: reelle Zahlen
- $\lg = \log_2$; $\ln = \log_e$

Folgen

$a_n = (-1)^n$

- (Reelle) *Zahlenfolge* $(a_n)_{n \in \mathbb{N}_{\geq 1}}$
  formal eine Funktion $a : \mathbb{N}_{\geq 1} \to \mathbb{R}$
  schreiben auch "$a_1, a_2, a_3, \ldots$"
- *Grenzwert* einer Folge
  $\lim\limits_{n \to \infty} a_n = a^*$ genau dann, wenn $\boxed{\forall \varepsilon > 0\ \exists n_0 \in \mathbb{N}_{\geq 1}\ \forall n \geq n_0 : |a_n - a^*| < \varepsilon}$
- Nicht jede Folge hat einen Grenzwert, aber wenn er existiert, dann ist er eindeutig.

Folgen – Beispiel

$$\boxed{\forall \varepsilon > 0\ \exists n_0 \in \mathbb{N}_{\geq 1}\ \forall n \geq n_0 : |a_n - a^*| < \varepsilon}$$

Es sei die Folge $(a_n)_{n \in \mathbb{N}_{\geq 1}}$ mit $a_n = \dfrac{1}{n}$ gegeben.

Es gibt: $\lim\limits_{n \to \infty} a_n = 0$

$\varepsilon > 0$ beliebig. Wählen $n_0 := \left\lceil \frac{1}{\varepsilon} \right\rceil + 1$

$\Rightarrow$ für $n \geq n_0$ beliebig gilt:

$$|a_n - 0| = \frac{1}{n} \underset{n \geq n_0}{\leq} \frac{1}{n_0} = \frac{1}{\left\lceil \frac{1}{\varepsilon} \right\rceil + 1}$$

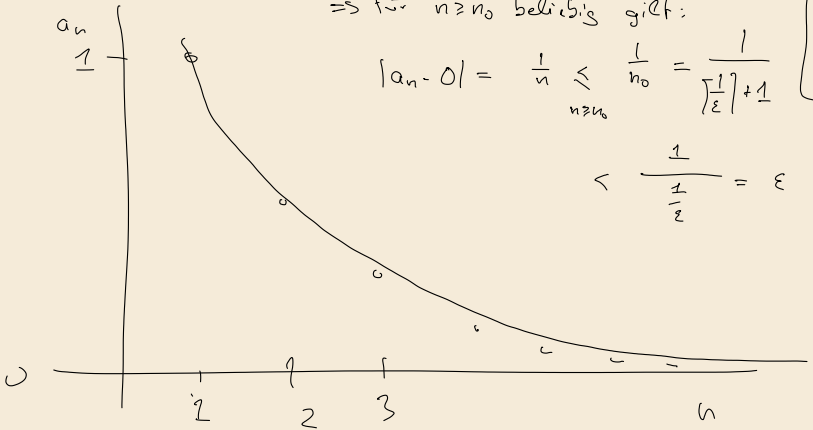
$$< \frac{1}{\frac{1}{\varepsilon}} = \varepsilon \qquad \square.$$

N.R.

$|a_n - 0| < \varepsilon$

$= \frac{1}{n} \overset{!}{<} \varepsilon$

$n > \frac{1}{\varepsilon}$ tut's



Grenzwerte von Funktionen

$f : \mathbb{R} \to \mathbb{R}$

"ε-δ-Kriterium"

$$\lim_{x \to \xi} f(x) = f^* \text{ gdw } \forall \varepsilon > 0 \; \exists \delta > 0 \; \forall x \in \mathbb{R} : |x - \xi| < \delta \implies |f(x) - f^*| < \varepsilon$$

Limes superior, Limes inferior

- Supremum ist die kleinste obere Schranke
  Für $M \subseteq \mathbb{R}$ ist $\sup M = \min\{b \in \mathbb{R} : \forall x \in M : x \leq b\}$

$$M = \left\{ 1 - \frac{1}{n} : n \in \mathbb{N}_0 \right\}$$

$$x \in M \Rightarrow x \leq 1$$

$$\sup M = 1$$

Limes superior, Limes inferior

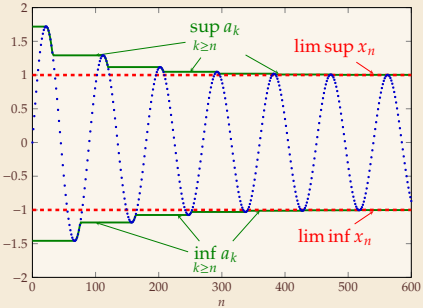
- Supremum ist die kleinste obere Schranke
  Für $M \subseteq \mathbb{R}$ ist $\sup M = \min\{b \in \mathbb{R} \,:\, \forall x \in M : x \leq b\}$

$\inf\left\{\frac{1}{n} \,;\, n \in \mathbb{N}\right\} = 0$

- Analog ist Infimum die größte untere Schranke
- Beide müssen nicht existieren ($+\infty$ bzw. $-\infty$); wenn sie existieren, sind sie eindeutig.

Limes superior, Limes inferior

- Supremum ist die kleinste obere Schranke
  Für $M \subseteq \mathbb{R}$ ist $\sup M = \min\{b \in \mathbb{R} \,:\, \forall x \in M : x \leq b\}$
- Analog ist Infimum die größte untere Schranke
- Beide müssen nicht existieren ($+\infty$ bzw. $-\infty$); wenn sie existieren, sind sie eindeutig.



- Für Folge $(a_n)_{n \in \mathbb{N}_{\geq 1}}$ ist der *Limes superior*:

$$\boxed{\limsup_{n \to \infty} a_n \;=\; \inf\Big\{\sup\big\{a_k : k \geq n\big\} : n \in \mathbb{N}_{\geq 1}\Big\}}$$

Äquivalent: größter Häufungspunkt von (a_n);
beste obere Schranke für a_n
für genügend große n

- Analog Limes inferior: $\liminf\limits_{n \to \infty} a_n \;=\; \sup\Big\{\inf\big\{a_k : k \geq n\big\} : n \in \mathbb{N}_{\geq 1}\Big\}$

3.3 Asymptotische Notation

Clicker Question

Ist $n^{0.1} = O\big((\log(n))^{10}\big)$?

- **A** O?? nie gesehen
- **B** Ja (bin mir sicher).
- **C** Nein (bin mir sicher).
- **D** Denke Ja
- **E** Denke Nein
- **F** keine Ahnung

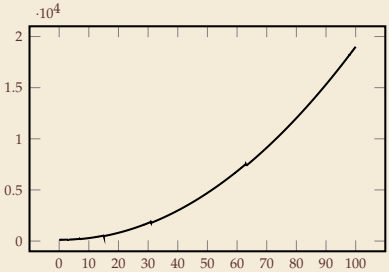
→ sli.do/cs210

Asymptotische Vergleiche

Asymptotik = Approximation um ∞

Beispiel: Betrachte Funktion $f(n)$ mit

$2n^2 - 3n\lfloor\log_2(n+1)\rfloor + 7n - 3\lfloor\log_2(n+1)\rfloor + 120$

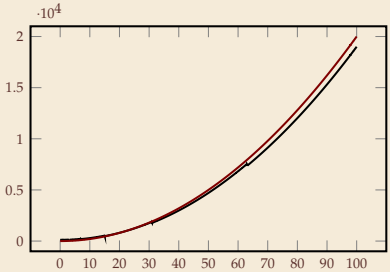


Asymptotische Vergleiche

Asymptotik = Approximation um ∞

Beispiel: Betrachte Funktion $f(n)$ mit

$$2n^2 - 3n\lfloor\log_2(n+1)\rfloor + 7n - 3\lfloor\log_2(n+1)\rfloor + 120 \sim 2n^2$$



Asymptotische Approximationen in der Analyse von Algorithmen

- abstrahiert von unwichtigen Details
- vereinfacht die Analyse selbst
- ermöglicht oft erst konklusiven Vergleich

Asymptotische Notation – Formale Definition

- **"Tilde Notation":** $f(n) \sim g(n)$ gdw $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 1$

  genau dann, wenn (*iff*)

  „f und g sind *asymptotisch äquivalent*"

Asymptotische Notation – Formale Definition

- **"Tilde Notation":** $f(n) \sim g(n)$ gdw $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 1$

  genau dann, wenn (*iff*)

  „f und g sind *asymptotisch äquivalent*"

- **"Big-Oh Notation":** $f(n) \in O\big(g(n)\big)$ gdw $\left|\frac{f(n)}{g(n)}\right|$ beschränkt für $n \geq n_0$

  schreiben auch '=' stattdessen

  brauchen Supremum da Limes evtl. nicht existiert!

  gdw $\lim_{n\to\infty} \sup \left|\frac{f(n)}{g(n)}\right| < \infty$

Varianten:

- $f(n) \in \Omega\big(g(n)\big)$ gdw $g(n) \in O\big(f(n)\big)$

  "Big-Omega"

- $f(n) \in \Theta\big(g(n)\big)$ gdw $f(n) \in O\big(g(n)\big)$ **und** $f(n) \in \Omega\big(g(n)\big)$

  "Big-Theta"

Asymptotische Notation – Formale Definition

- **"Tilde Notation":** $f(n) \sim g(n)$ gdw $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 1$.

  genau dann, wenn (*iff*)

  „f und g sind *asymptotisch äquivalent*"

- **"Big-Oh Notation":** $f(n) \in O\big(g(n)\big)$ gdw $\left|\frac{f(n)}{g(n)}\right|$ beschränkt für $n \geq n_0$

  schreiben auch '=' stattdessen

  gdw $\limsup_{n\to\infty} \left|\frac{f(n)}{g(n)}\right| < \infty$

  brauchen Supremum da Limes evtl. nicht existiert!

  Varianten:

  - $f(n) \in \Omega\big(g(n)\big)$ gdw $g(n) \in O\big(f(n)\big)$

    "Big-Omega"

  - $f(n) \in \Theta\big(g(n)\big)$ gdw $f(n) \in O\big(g(n)\big)$ **und** $f(n) \in \Omega\big(g(n)\big)$

    "Big-Theta"

- **"Little-Oh Notation":** $f(n) \in o\big(g(n)\big)$ gdw $\lim_{n\to\infty} \left|\frac{f(n)}{g(n)}\right| = 0$

  analog: $f(n) \in \omega\big(g(n)\big)$ falls $|f(n)/g(n)| \to \infty$

(Vorteil dieser Definition: Funktioniert genauso für $f, g : \mathbb{R} \to \mathbb{R}$ und für andere Grenzprozesse als $n \to \infty$)

Asymptotische Notation – Intuition

"≤"

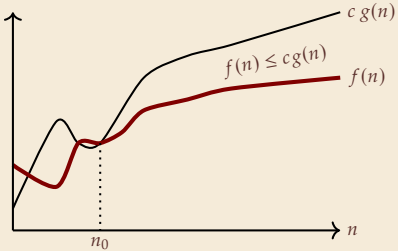
- $f(n) = O(g(n))$: $f(n)$ ist **höchstens** $g(n)$
  bis auf konstante Faktoren
  und
  für hinreichend große n

"≥"

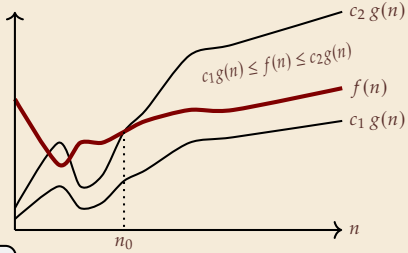
$f(n) = \Omega(g(n))$

"≪"

$f(n) = o(g(n))$



- $f(n) = \Theta(g(n))$: $f(n)$ ist **gleich** $g(n)$
  bis auf konstante Faktoren
  und
  für hinreichend große n



Plots können irreführend sein!

Beispiel

O-Notation – Beispiel

$f(n) = 3n^3 + 2n^2, \quad g(n) = n^3$

$$\frac{f(n)}{g(n)} = \frac{3n^3 + 2n^2}{n^3} = 3 + \underbrace{\frac{2}{n}}_{\to 0}$$

$$\lim_{n \to \infty} \frac{f(n)}{g(n)} = 3$$

$f \not\sim g$

$f(n) = O(g(n))$ da 3 beschränkt

$g(n) \overset{?}{=} O(f(n))$ ✓ $\quad \frac{g(n)}{f(n)} \to \frac{1}{3}$

$\Rightarrow f(n) = \Omega(g(n))$

$\Rightarrow f(n) = \Theta(g(n))$

3.4 Rechenregeln für Asymptotiken

O-Notation – Beispiel 2

$$f(n) \in o(g(n)) \quad \text{gdw} \quad \lim_{n\to\infty} \left|\frac{f(n)}{g(n)}\right| = 0$$

$n \ln(n) = o(n^2)$ ✓

$\ln = \log_e$

$$\lim_{n\to\infty} \frac{n \ln(n)}{n^2} = \lim_{n\to\infty} \frac{\ln(n)}{n} = \text{“}\frac{\infty}{\infty}\text{”}$$

$f(n) = \ln(n)$ $\qquad f'(n) = \frac{1}{n}$ ✗

$g(n) = n$ $\qquad g'(n) = 1$ ✗

$$\lim_{n\to\infty} \frac{f'(n)}{g'(n)} = \lim_{n\to\infty} \frac{1}{n} = 0$$

$$\Rightarrow \quad \lim_{n\to\infty} \frac{n \ln(n)}{n^2} = \lim_{n\to\infty} \frac{f'(n)}{g'(n)} = \lim_{n\to\infty} \frac{1}{n} = 0$$

l'Hospital

Regel von de l'Hospital

Bei Grenzwerten der Form "$\frac{\infty}{\infty}$" bzw. "$\frac{0}{0}$" kann man oft folgende Regel anwenden.

Theorem 3.1 (de l'Hospital)

Gegeben Funktionen $f, g : \mathbb{R}_{>n_0} \to \mathbb{R}$, die

1. *differenzierbar* auf $\mathbb{R}_{>n_0}$ sind,
2. $\forall x \in \mathbb{R}_{>n_0} : g'(x) \neq 0$,
3. $\lim\limits_{x\to\infty} |f(x)| = \lim\limits_{x\to\infty} |g(x)| \in \{0, \infty\}$, und
4. $\lim_{x\to\infty} \frac{f'(x)}{g'(x)}$ existiert

Dann ist $\lim\limits_{n\to\infty} \frac{f(n)}{g(n)} = \lim\limits_{n\to\infty} \frac{f'(n)}{g'(n)}$.

Ausnahme von der Regel „log wie ohne Basis"

in $\Theta(\log n) = \Theta(\log_2(n)) = \Theta(\log_e(n))$

$$\log_b(n) = \frac{\ln(n)}{\underbrace{\ln(b)}_{=c}} = c \cdot \ln(n)$$

$$\frac{\log_b(n)}{\ln(n)} = \frac{1}{c} \quad \Rightarrow \quad \log_b(n) = \Theta(\ln(n))$$

Clicker Question

$$\lim_{n\to\infty} \frac{n^{0.1}}{\ln^{10}(n)} = \text{"}\frac{\infty}{\infty}\text{"} \qquad \frac{d}{dn} n^{0.1} = 0.1 \cdot n^{-0.9}$$

$$\frac{d}{dn} \ln^{10}(n) = 10 \ln^{9}(n) \cdot \frac{1}{n} \qquad \frac{\frac{d}{dn} n^{0.1}}{\frac{d}{dn} \ln^{10}(n)} = \frac{n^{0.1}}{100 \ln^{9}(n)}.$$

$\vdots$

10 mal

l'Hospital

$\to \infty$

Ist $n^{0.1} = O\big((\log(n))^{10}\big)$?

A O?? nie gesehen

B Ja (bin mir sicher).

C Nein (bin mir sicher).

D Denke Ja

E Denke Nein

F keine Ahnung

$$\forall c, d > 0 \qquad n^{c} = \omega\left(\log^{d}(n)\right)$$

→ sli.do/cs210

Clicker Question

Ist $n^{0.1} = O\big((\log(n))^{10}\big)$?

- **A** ~~O?? nie gesehen~~
- **B** ~~Ja (bin mir sicher).~~
- **C** Nein (bin mir sicher). ✓
- **D** ~~Denke Ja~~
- **E** ~~Denke Nein~~
- **F** ~~keine Ahnung~~

→ sli.do/cs210

Rechenregeln

Lemma 3.2 (Rechenregeln für asymptotische Notationen)

Es gilt:

(a) $f = O(f)$, $f = \Theta(f)$, $f = \Omega(f)$ *(Reflexivität)*.

(b) $c \cdot O(f) = O(f)$ für c konstant.

$f \notin o(f)$

$f \notin \omega(f)$

Rechenregeln

Lemma 3.2 (Rechenregeln für asymptotische Notationen)

Es gilt:

(a) $f = O(f), \quad f = \Theta(f), \quad f = \Omega(f)$ *(Reflexivität)*.

(b) $c \cdot O(f) = O(f)$ für c konstant.

(c) $O(f) + O(g) = O(f + g) = O(\max(f, g))$.

(d) $O(O(f)) = O(f)$.

(e) $O(f) \cdot O(g) = O(f \cdot g)$.

Rechenregeln

Lemma 3.2 (Rechenregeln für asymptotische Notationen)

Es gilt:

(a) $f = O(f)$, $f \in \Theta(f)$, $f = \Omega(f)$ *(Reflexivität)*.

(b) $c \cdot O(f) = O(f)$ für c konstant.

(c) $O(f) + O(g) = O(f+g) = O(\max(f,g))$.

(d) $O(O(f)) = O(f)$.

(e) $O(f) \cdot O(g) = O(f \cdot g)$.

(f) Falls $f \sim g$, dann gilt $f = g \cdot (1 \pm o(1))$.

(g) $f = O(g) \leftrightarrow g = \Omega(f)$ *(Schiefsymmetrie)*,

$f = \Theta(g) \leftrightarrow g = \Theta(f)$ *(Symmetrie)*.

(h) $\Omega(f) + \Omega(g) = \Omega(\min(f,g))$.

$$O(f) = \left\{ h : \lim_{n\to\infty} \left| \frac{h(n)}{f(n)} \right| < \infty \right\}$$

$$O(g) = \left\{ h : \lim_{n\to\infty} \left| \frac{h(n)}{g(n)} \right| < \infty \right\}$$

$$O(f) + O(g) = \left\{ h : h(n) = h_1(n) + h_2(n) \wedge h_1 \in O(f) \wedge h_2 \in O(g) \right\}$$

$$1 \pm o(1) = \left\{ h : h(n) = 1 + h_1(n),\ h_1 \in o(1) \vee h(n) = 1 - h_2(n),\ h_2 \in o(1) \right\}$$

Rechenregeln

Lemma 3.2 (Rechenregeln für asymptotische Notationen)

Es gilt:

(a) $f = O(f), \quad f = \Theta(f), \quad f = \Omega(f)$ *(Reflexivität)*.

(b) $c \cdot O(f) = O(f)$ für c konstant.

(c) $O(f) + O(g) = O(f + g) = O(\max(f, g))$.

(d) $O(O(f)) = O(f)$.

(e) $O(f) \cdot O(g) = O(f \cdot g)$.

(f) Falls $f \sim g$, dann gilt $f = g \cdot (1 \pm o(1))$.

(g) $f = O(g) \leftrightarrow g = \Omega(f)$ *(Schiefsymmetrie)*,

$f = \Theta(g) \leftrightarrow g = \Theta(f)$ *(Symmetrie)*.

(h) $\Omega(f) + \Omega(g) = \Omega(\min(f, g))$.

(i) $f = \circ(g) \wedge g = \circ(h) \rightarrow f = \circ(h)$ für $\circ \in \{O, o, \Omega, \omega, \Theta\}$ *(Transitivität)*.

O-Notation – Quantoren-Definition

Für alle asymptotischen Notationen $f(n) = O(g(n))$: Grenzwert des Quotienten $\dfrac{f(n)}{g(n)}$

Alternative Definition (EAA Buch) (äquivalent für $f : \mathbb{N}_{\geq 0} \to \mathbb{R}_{\geq 0}$)

Definition 1.16 (Asymptotische Notation [Landau-Klassen]):
Sei $\mathcal{F} := \mathsf{Abb}(\mathbb{N}_0, \mathbb{R}_0^+)$ die Menge der Abbildungen von der Menge der natürlichen Zahlen inklusive der Null in die Menge der nicht-negativen reellen Zahlen. Sei ferner $g \in \mathcal{F}$. Wir definieren die folgenden Mengen von Funktionen:

1.) $\mathcal{O}(g) := \{f \in \mathcal{F} \mid \underline{(\exists c > 0)(\exists n_0 \in \mathbb{N}_0)(\forall n \geq n_0)}\ (f(n) \leq c \cdot g(n))\}$,

2.) $\Omega(g) := \{f \in \mathcal{F} \mid (\exists c > 0)(\exists n_0 \in \mathbb{N}_0)(\forall n \geq n_0)\ (f(n) \geq c \cdot g(n))\}$,

3.) $\Theta(g) := \mathcal{O}(g) \cap \Omega(g)$,

4.) $o(g) := \{f \in \mathcal{F} \mid (\forall c > 0)(\exists n_0 \in \mathbb{N}_0)(\forall n \geq n_0)\ (f(n) \leq c \cdot g(n))\}$,

5.) $\omega(g) := \{f \in \mathcal{F} \mid (\forall c > 0)(\exists n_0 \in \mathbb{N}_0)(\forall n \geq n_0)\ (f(n) \geq c \cdot g(n))\}$,

6.) $\sim(g) := \{f \in \mathcal{F} \mid (\forall \varepsilon > 0)(\exists n_0 \in \mathbb{N}_0)(\forall n \geq n_0)\ \left(\left|\frac{f(n)}{g(n)} - 1\right| \leq \varepsilon\right)\}$.
Hier nehmen wir an, dass $g(n) = 0$ höchstens für endlich viele n gilt.

(Beachte die unterschiedlichen Quantoren bei c bzw. ε!) ◀

3.5 Rekursiv definierte Folgen

Clicker Question

Kennen Sie die Türme von Hanoi?

A Absolut, könnte ich erklären.

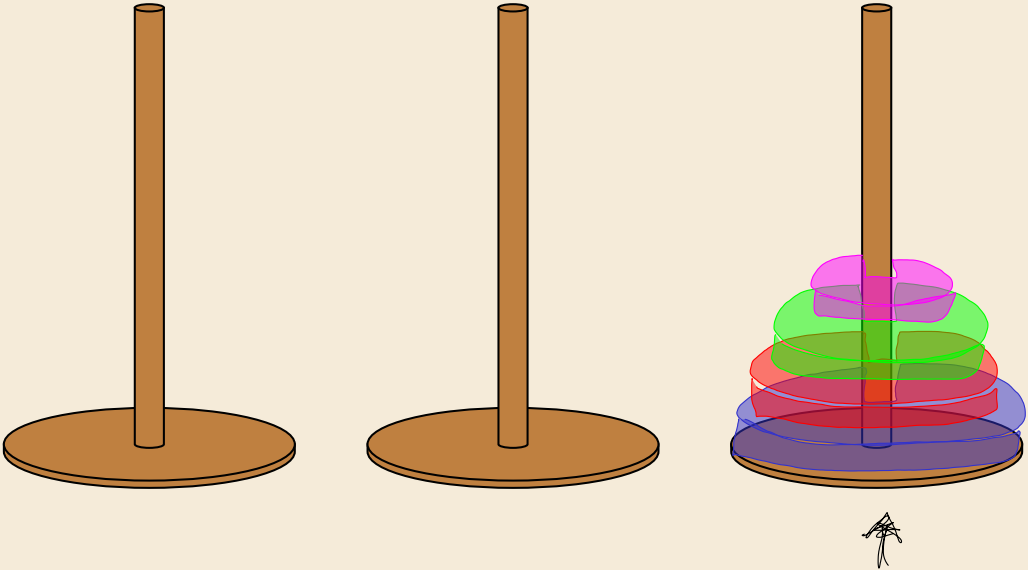
B Kam wo vor, aber ich erinnere mich nicht so ganz.

C Ziemlich sicher noch nie gehört.

D War noch nie in Vietnam.

→ sli.do/cs210

Die “Türme von Hanoi”



Rekursive Lösung

```
procedure towersOfHanoi(s, t, b, n)
    // Move topmost n pieces from s to t
    // Assumes the topmost n disks are on pile s
    if n == 0 return
    towersOfHanoi(s, b, t, n − 1)
    move disk n from s to t
    towersOfHanoi(b, t, s, n − 1)
```

s: Quellstab (source)

t: Zielstab (target)

b: Hilfsstab (buffer)

towersOfHanoi(1, 3, 2, n = 4)

Rekursive Lösung

```
procedure towersOfHanoi(s, t, b, n)
    // Move topmost n pieces from s to t
    // Assumes the topmost n disks are on pile s
    if n == 0 return
    towersOfHanoi(s, b, t, n − 1)
    move disk n from s to t
    towersOfHanoi(b, t, s, n − 1)
```

s: Quellstab (source)

t: Zielstab (target)

b: Hilfsstab (buffer)

- einfacher Code
- landet gezielt auf korrektem Stab!

https://upload.wikimedia.org/wikipedia/commons/2/20/Tower_of_Hanoi_recursion_SMIL.svg

Rekursive Lösung

```
1 procedure towersOfHanoi(s, t, b, n)
2     // Move topmost n pieces from s to t
3     // Assumes the topmost n disks are on pile s
4     if n == 0 return
5     towersOfHanoi(s, b, t, n − 1)
6     move disk n from s to t
7     towersOfHanoi(b, t, s, n − 1)
```

s: Quellstab (source)

t: Zielstab (target)

b: Hilfsstab (buffer)

Anzahl Bewegungen

- Nur Zeile 6 bewegt eine Scheibe!

- einfacher Code
- landet gezielt auf korrektem Stab!

https://upload.wikimedia.org/wikipedia/commons/2/20/Tower_of_Hanoi_recursion_SMIL.svg

- aber wie lange braucht der Code denn?

Rekursive Lösung

```
1 procedure towersOfHanoi(s, t, b, n)
2     // Move topmost n pieces from s to t
3     // Assumes the topmost n disks are on pile s
4     if n == 0 return
5     towersOfHanoi(s, b, t, n − 1)
6     move disk n from s to t
7     towersOfHanoi(b, t, s, n − 1)
```

s: Quellstab (source)

t: Zielstab (target)

b: Hilfsstab (buffer)

- einfacher Code
- landet gezielt auf korrektem Stab!

https://upload.wikimedia.org/wikipedia/commons/2/20/Tower_of_Hanoi_recursion_SMIL.svg

- aber wie lange braucht der Code denn?

Anzahl Bewegungen

- Nur Zeile 6 bewegt eine Scheibe! ... aber natürlich auch rekursiv aufgerufen.
  - Für $n = 0$ keine weitere Bewegung; sonst eine Bewegung und 2 mal die Bewegungen für $n - 1$

Rekursive Lösung

```
1 procedure towersOfHanoi(s, t, b, n)
2     // Move topmost n pieces from s to t
3     // Assumes the topmost n disks are on pile s
4     if n == 0 return
5     towersOfHanoi(s, b, t, n − 1)
6     move disk n from s to t
7     towersOfHanoi(b, t, s, n − 1)
```

s: Quellstab (source)

t: Zielstab (target)

b: Hilfsstab (buffer)

- einfacher Code
- landet gezielt auf korrektem Stab!

https://upload.wikimedia.org/wikipedia/commons/2/20/Tower_of_Hanoi_recursion_SMIL.svg

- aber wie lange braucht der Code denn?

Anzahl Bewegungen

- Nur Zeile 6 bewegt eine Scheibe! ... aber natürlich auch rekursiv aufgerufen.
  - Für $n = 0$ keine weitere Bewegung; sonst eine Bewegung und 2 mal die Bewegungen für $n - 1$

$\rightsquigarrow$ *Rekursionsgleichung / Rekurrenz*: $T : \mathbb{N}_{\geq 0} \to \mathbb{N}_{\geq 0}$ $T(n) = \#$ Bewegungen für n Scheiben

$T(0) = 0,$

Rekursive Lösung

```
1 procedure towersOfHanoi(s, t, b, n)
2     // Move topmost n pieces from s to t
3     // Assumes the topmost n disks are on pile s
4     if n == 0 return
5     towersOfHanoi(s, b, t, n − 1)
6     move disk n from s to t
7     towersOfHanoi(b, t, s, n − 1)
```

s: Quellstab (source)

t: Zielstab (target)

b: Hilfsstab (buffer)

- einfacher Code
- landet gezielt auf korrektem Stab!

https://upload.wikimedia.org/wikipedia/commons/2/20/Tower_of_Hanoi_recursion_SMIL.svg

- aber wie lange braucht der Code denn?

Anzahl Bewegungen

- Nur Zeile 6 bewegt eine Scheibe! ... aber natürlich auch rekursiv aufgerufen.
  - Für $n = 0$ keine weitere Bewegung; sonst eine Bewegung und 2 mal die Bewegungen für $n - 1$

⇝ *Rekursionsgleichung / Rekurrenz*: $T : \mathbb{N}_{\geq 0} \to \mathbb{N}_{\geq 0}$ $T(n) = \#$ Bewegungen für n Scheiben

$$T(0) = 0, \qquad T(n) = T(n-1) + 1 + T(n-1) = 2T(n-1) + 1 \qquad (n \geq 1)$$

Rekursion und Rekurrenzen

$$\boxed{T(0) \;=\; 0, \quad T(n) \;=\; 2T(n-1)+1 \quad (n \geq 1)}$$

1. **Werte für kleine *n***

n	$T(n)$
0	0

Rekursion und Rekurrenzen

$$\boxed{T(0) \;=\; 0, \quad T(n) \;=\; 2T(n-1)+1 \quad (n \geq 1)}$$

1. **Werte für kleine *n***

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 \;=\; 1$

Rekursion und Rekurrenzen

$$\boxed{T(0) \;=\; 0, \quad T(n) \;=\; 2T(n-1)+1 \quad (n \geq 1)}$$

1. **Werte für kleine *n***

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 \;=\; 1$
2	$2 \cdot 1 + 1 \;=\; 3$

Rekursion und Rekurrenzen

$$\boxed{T(0) \;=\; 0, \quad T(n) \;=\; 2T(n-1)+1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 \;=\; 1$
2	$2 \cdot 1 + 1 \;=\; 3$
3	$2 \cdot 3 + 1 \;=\; 7$
4	$2 \cdot 7 + 1 \;=\; 15$
5	$2 \cdot 15 + 1 \;=\; 31$
6	$2 \cdot 31 + 1 \;=\; 63$

(Informatiker erkennen Zweierpotenzen 😉)

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$T(n)$

Rekursion und Rekurrenzen

$$\boxed{T(0) \;=\; 0, \quad T(n) \;=\; 2T(n-1)+1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 \;=\; 1$
2	$2 \cdot 1 + 1 \;=\; 3$
3	$2 \cdot 3 + 1 \;=\; 7$
4	$2 \cdot 7 + 1 \;=\; 15$
5	$2 \cdot 15 + 1 \;=\; 31$
6	$2 \cdot 31 + 1 \;=\; 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$T(n) \;=\; 2T(n-1)+1$$

Rekursion und Rekurrenzen

$$\boxed{T(0) \;=\; 0, \quad T(n) \;=\; 2T(n-1)+1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 \;=\; 1$
2	$2 \cdot 1 + 1 \;=\; 3$
3	$2 \cdot 3 + 1 \;=\; 7$
4	$2 \cdot 7 + 1 \;=\; 15$
5	$2 \cdot 15 + 1 \;=\; 31$
6	$2 \cdot 31 + 1 \;=\; 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned} T(n) \;&=\; 2T(n-1)+1 \\ &=\; 2\big(2T(n-2)+1\big)+1 \end{aligned}$$

Rekursion und Rekurrenzen

$$\boxed{T(0) \;=\; 0, \quad T(n) \;=\; 2T(n-1)+1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 \;=\; 1$
2	$2 \cdot 1 + 1 \;=\; 3$
3	$2 \cdot 3 + 1 \;=\; 7$
4	$2 \cdot 7 + 1 \;=\; 15$
5	$2 \cdot 15 + 1 \;=\; 31$
6	$2 \cdot 31 + 1 \;=\; 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned} T(n) \;&=\; 2T(n-1)+1 \\ &=\; 2\big(2T(n-2)+1\big)+1 \\ &=\; 2^{\mathbf{2}}T(n-\mathbf{2})+2+1 \end{aligned}$$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2\big(2T(n-2) + 1\big) + 1 \\ &= 2^{2}T(n-2) + 2 + 1 \\ &= 2^{3}T(n-3) + 2^2 + 2 + 1 \end{aligned}$$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 \\
&= 2\big(2T(n-2) + 1\big) + 1 \\
&= 2^{\mathbf{2}}T(n-\mathbf{2}) + 2 + 1 \\
&= 2^{\mathbf{3}}T(n-\mathbf{3}) + 2^2 + 2 + 1 \\
&= \cancel{2^{\boldsymbol{n}}T(n-\boldsymbol{n})} + \\
&\quad 2^{n-1} + \cdots + 2 + 1
\end{aligned}$$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 \\
&= 2\big(2T(n-2) + 1\big) + 1 \\
&= 2^{\mathbf{2}}T(n-\mathbf{2}) + 2 + 1 \\
&= 2^{\mathbf{3}}T(n-\mathbf{3}) + 2^2 + 2 + 1 \\
&= 2^{\boldsymbol{n}}T(n-\boldsymbol{n}) + \\
&\quad 2^{n-1} + \cdots + 2 + 1 \\
&= \sum_{i=0}^{n-1} 2^i
\end{aligned}$$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 \\
&= 2\big(2T(n-2) + 1\big) + 1 \\
&= 2^{\mathbf{2}}T(n-\mathbf{2}) + 2 + 1 \\
&= 2^{\mathbf{3}}T(n-\mathbf{3}) + 2^2 + 2 + 1 \\
&= 2^{\boldsymbol{n}}T(n-\boldsymbol{n}) + \\
&\quad 2^{n-1} + \cdots + 2 + 1 \\
&= \sum_{i=0}^{n-1} 2^i = 2^n - 1
\end{aligned}$$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 \\
&= 2\big(2T(n-2) + 1\big) + 1 \\
&= 2^{\mathbf{2}}T(n-\mathbf{2}) + 2 + 1 \\
&= 2^{\mathbf{3}}T(n-\mathbf{3}) + 2^2 + 2 + 1 \\
&= 2^{\boldsymbol{n}}T(n-\boldsymbol{n}) + \\
&\quad 2^{n-1} + \cdots + 2 + 1 \\
&= \sum_{i=0}^{n-1} 2^i = 2^n - 1
\end{aligned}$$

$\rightsquigarrow$ *"educated guess"*:

$$\boxed{T(n) = \mathbf{2^n - 1}}$$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 \\
&= 2\big(2T(n-2) + 1\big) + 1 \\
&= 2^{\mathbf{2}}T(n-\mathbf{2}) + 2 + 1 \\
&= 2^{\mathbf{3}}T(n-\mathbf{3}) + 2^2 + 2 + 1 \\
&= 2^{\boldsymbol{n}}T(n-\boldsymbol{n}) + \\
&\quad 2^{n-1} + \cdots + 2 + 1 \\
&= \sum_{i=0}^{n-1} 2^i = 2^n - 1
\end{aligned}$$

$\rightsquigarrow$ *"educated guess":*

$$\boxed{T(n) = \mathbf{2^n - 1}}$$

3. Per Induktion beweisen

Lemma 3.3 (Türme von Hanoi)

$T(n) = 2^n - 1.$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 \\
&= 2\big(2T(n-2) + 1\big) + 1 \\
&= 2^{\mathbf{2}}T(n-\mathbf{2}) + 2 + 1 \\
&= 2^{\mathbf{3}}T(n-\mathbf{3}) + 2^2 + 2 + 1 \\
&= 2^{\boldsymbol{n}}T(n-\boldsymbol{n}) + \\
&\quad 2^{n-1} + \cdots + 2 + 1 \\
&= \sum_{i=0}^{n-1} 2^i = 2^n - 1
\end{aligned}$$

$\rightsquigarrow$ *"educated guess"*:

$$\boxed{T(n) = \mathbf{2^n - 1}}$$

3. Per Induktion beweisen

Lemma 3.3 (Türme von Hanoi)

$T(n) = 2^n - 1.$

Beweis:

Per Induktion über n.

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2\big(2T(n-2) + 1\big) + 1 \\ &= 2^2 T(n-2) + 2 + 1 \\ &= 2^3 T(n-3) + 2^2 + 2 + 1 \\ &= 2^n T(n-n) + \\ &\quad 2^{n-1} + \cdots + 2 + 1 \\ &= \sum_{i=0}^{n-1} 2^i = 2^n - 1 \end{aligned}$$

$\rightsquigarrow$ *"educated guess"*:

$$\boxed{T(n) = 2^n - 1}$$

3. Per Induktion beweisen

Lemma 3.3 (Türme von Hanoi)

$T(n) = 2^n - 1.$ für alle $n \in \mathbb{N}_0$

Beweis:

Per Induktion über n.

- Induktionsanfang (IA): $n = 0$

  $T(0) = 0$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned}
T(n) &= 2T(n-1) + 1 \\
&= 2\big(2T(n-2) + 1\big) + 1 \\
&= 2^2 T(n-2) + 2 + 1 \\
&= 2^3 T(n-3) + 2^2 + 2 + 1 \\
&= 2^n T(n-n) + \\
&\quad 2^{n-1} + \cdots + 2 + 1 \\
&= \sum_{i=0}^{n-1} 2^i = 2^n - 1
\end{aligned}$$

$\rightsquigarrow$ *"educated guess"*:

$$\boxed{T(n) = 2^n - 1}$$

3. Per Induktion beweisen

Lemma 3.3 (Türme von Hanoi)

$T(n) = 2^n - 1.$

Beweis:

Per Induktion über n.

- Induktionsanfang (IA): $n = 0$

  $T(0) = 0 = 2^0 - 1$ ✓

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1)+1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned} T(n) &= 2T(n-1)+1 \\ &= 2\big(2T(n-2)+1\big)+1 \\ &= 2^{\mathbf{2}}T(n-\mathbf{2})+2+1 \\ &= 2^{\mathbf{3}}T(n-\mathbf{3})+2^2+2+1 \\ &= 2^{\boldsymbol{n}}T(n-\boldsymbol{n})+ \\ &\quad 2^{n-1}+\cdots+2+1 \\ &= \sum_{i=0}^{n-1} 2^i = 2^n - 1 \end{aligned}$$

$\rightsquigarrow$ *"educated guess"*:

$$\boxed{T(n) = \mathbf{2^n - 1}}$$

3. Per Induktion beweisen

Lemma 3.3 (Türme von Hanoi)

$T(n) = 2^n - 1.$

Beweis:

Per Induktion über n.

- Induktionsanfang (IA): $n = 0$

  $T(0) = 0 = 2^0 - 1$ ✓
- Induktionsvoraussetzung (IV):

  $\forall n' \leq n \ : \ T(n') = 2^{n'} - 1$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2\big(2T(n-2) + 1\big) + 1 \\ &= 2^2 T(n-2) + 2 + 1 \\ &= 2^3 T(n-3) + 2^2 + 2 + 1 \\ &= 2^n T(n-n) + \\ &\quad 2^{n-1} + \cdots + 2 + 1 \\ &= \sum_{i=0}^{n-1} 2^i = 2^n - 1 \end{aligned}$$

$2^{n+1} - 1$

$\rightsquigarrow$ *"educated guess"*:

$$\boxed{T(n) = 2^n - 1}$$

3. Per Induktion beweisen

Lemma 3.3 (Türme von Hanoi)

$T(n) = 2^n - 1.$

Beweis:

Per Induktion über n.

- Induktionsanfang (IA): $n = 0$
  $T(0) = 0 = 2^0 - 1$ ✓
- Induktionsvoraussetzung (IV):
  $\forall n' \leq n : T(n') = 2^{n'} - 1$
- Induktionsschritt (IS): $n \to n + 1$
  $T(n+1) = 2T(n) + 1$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2\big(2T(n-2) + 1\big) + 1 \\ &= 2^{\mathbf{2}}T(n-\mathbf{2}) + 2 + 1 \\ &= 2^{\mathbf{3}}T(n-\mathbf{3}) + 2^2 + 2 + 1 \\ &= 2^{\boldsymbol{n}}T(n-\boldsymbol{n}) + \\ &\quad 2^{n-1} + \cdots + 2 + 1 \\ &= \sum_{i=0}^{n-1} 2^i = 2^n - 1 \end{aligned}$$

$\rightsquigarrow$ *"educated guess"*:

$$\boxed{T(n) = \mathbf{2^n - 1}}$$

3. Per Induktion beweisen

Lemma 3.3 (Türme von Hanoi)

$T(n) = 2^n - 1.$

Beweis:

Per Induktion über n.

- Induktionsanfang (IA): $n = 0$

  $T(0) = 0 = 2^0 - 1$ ✓
- Induktionsvoraussetzung (IV):

  $\forall n' \leq n \ : \ T(n') = 2^{n'} - 1$
- Induktionsschritt (IS): $n \to n+1$

  $$\begin{aligned} T(n+1) &= 2T(n) + 1 \\ &\underset{\text{IV}}{=} 2\big(2^n - 1\big) + 1 \end{aligned}$$

Rekursion und Rekurrenzen

$$\boxed{T(0) = 0, \quad T(n) = 2T(n-1) + 1 \quad (n \geq 1)}$$

1. Werte für kleine n

n	$T(n)$
0	0
1	$2 \cdot 0 + 1 = 1$
2	$2 \cdot 1 + 1 = 3$
3	$2 \cdot 3 + 1 = 7$
4	$2 \cdot 7 + 1 = 15$
5	$2 \cdot 15 + 1 = 31$
6	$2 \cdot 31 + 1 = 63$

(Informatiker erkennen Zweierpotenzen 😉)

2. Lösung raten

(manchmal schwierig!!)

Oft hilfreich: einsetzen!

$$\begin{aligned} T(n) &= 2T(n-1) + 1 \\ &= 2\big(2T(n-2) + 1\big) + 1 \\ &= 2^2 T(n-2) + 2 + 1 \\ &= 2^3 T(n-3) + 2^2 + 2 + 1 \\ &= 2^n T(n-n) + \\ &\quad 2^{n-1} + \cdots + 2 + 1 \\ &= \sum_{i=0}^{n-1} 2^i = 2^n - 1 \end{aligned}$$

$\rightsquigarrow$ *"educated guess"*:

$$\boxed{T(n) = 2^n - 1}$$

3. Per Induktion beweisen

Lemma 3.3 (Türme von Hanoi)

$T(n) = 2^n - 1.$

Beweis:

Per Induktion über n.

- Induktionsanfang (IA): $n = 0$

  $T(0) = 0 = 2^0 - 1$ ✓

- Induktionsvoraussetzung (IV):

  $\forall n' \leq n : T(n') = 2^{n'} - 1$

- Induktionsschritt (IS): $n \to n+1$

$$\begin{aligned} T(n+1) &= 2T(n) + 1 \\ &\underset{\text{IV}}{=} 2\big(2^n - 1\big) + 1 \\ &= 2^{n+1} - 2 + 1 \\ &= 2^{n+1} - 1 \end{aligned}$$

■

$2^{64} \geq (2^{10})^6 = 10^{18}$

3.6 Das Master-Theorem

Clicker Question

Wie funktioniert Mergesort?

A Partitioniere Elemente um Median, sortiere dann rekursiv kleinere und größere Elemente.

B Sortiere rekursiv linke und rechte Hälfte und kombiniere die sortierten Hälften.

C Bilde sortierten Präfix, füge wiederholt nächstes Element an passende Stelle hinzu.

D Wähle wiederholt 2 Elemente und vertausche sie, falls sie nicht in der richtigen Reihenfolge stehen.

E Was fürn Ding?

→ sli.do/cs210

Clicker Question

Wie funktioniert Mergesort?

A ~~Partitioniere Elemente um Median, sortiere dann rekursiv kleinere und größere Elemente.~~

B Sortiere rekursiv linke und rechte Hälfte und kombiniere die sortierten Hälften. ✓

C ~~Bilde sortierten Präfix, füge wiederholt nächstes Element an passende Stelle hinzu.~~

D ~~Wähle wiederholt 2 Elemente und vertausche sie, falls sie nicht in der richtigen Reihenfolge stehen.~~

E ~~Was fürn Ding?~~

→ sli.do/cs210

Divide-and-Conquer Rekursionen

Wir werden eine Reihe von Algorithmen sehen die nach dem *Divide-&-Conquer-Prinzip* (*„Teile und herrsche“*) arbeiten.

Divide-and-Conquer Rekursionen

Wir werden eine Reihe von Algorithmen sehen die nach dem *Divide-&-Conquer-Prinzip* (*„Teile und herrsche“*) arbeiten.

Klassisches Beispiel: *Mergesort* (⇝ Unit 7)

0. Falls Eingabegröße $n \leq 1$, **fertig**.

1. *Teile* die Eingabe der Größe n in **2** Teile der Größe $\approx \frac{n}{2}$ auf.

2. Löse **rekursiv** die **2** Teilprobleme der Größe $\frac{n}{2}$.

3. **Kombiniere** die Lösungen der Teilprobleme zu einer Gesamtlösung.

(Wir kümmern uns hier (noch) nicht darum, wie/was das algorithmisch tut!)

Divide-and-Conquer Rekursionen

Wir werden eine Reihe von Algorithmen sehen die nach dem *Divide-&-Conquer-Prinzip* (*„Teile und herrsche“*) arbeiten.

Klassisches Beispiel: *Mergesort* (⇝ Unit 7)

0. Falls Eingabegröße $n \leq 1$, **fertig**.
1. *Teile* die Eingabe der Größe n in **2** Teile der Größe $\approx \frac{n}{2}$ auf.
2. Löse **rekursiv** die **2** Teilprobleme der Größe $\frac{n}{2}$.
3. **Kombiniere** die Lösungen der Teilprobleme zu einer Gesamtlösung.

(Wir kümmern uns hier (noch) nicht darum, wie/was das algorithmisch tut!)

Aber wir wollen Verfahren kennenlernen, wie man solche Algorithmen analysieren kann.

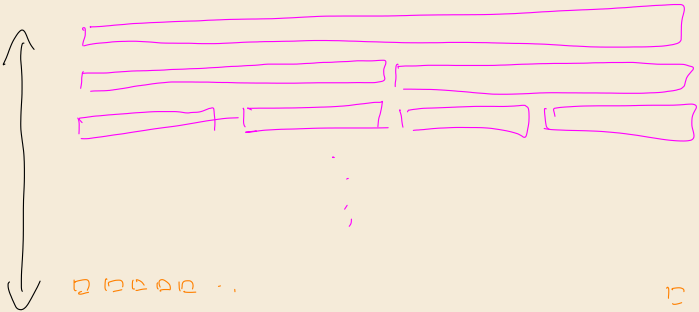
Laufzeit von Mergesort

$$\boxed{T(n) = \begin{cases} 0 & n \leq 1 \\ 2 \cdot T(n/2) + n & n \geq 2 \end{cases}}$$

Für $n = 2^k$, $k \in \mathbb{N}_0$

Rekursionsbaum-Methode für Mergesort

$$T(n) = \begin{cases} \mathbf{0} & n \leq 1 \\ \mathbf{2} \cdot T(n/\mathbf{2}) + \boldsymbol{n} & n \geq 2 \end{cases}$$



n

$2 \cdot \frac{n}{2} = n$

$4 \cdot \frac{n}{4} = n$

n

Die Master-Methode

- Allgemeines D&C-Verfahren
  - a rekursive Aufrufe (für Konstante $a > 0$)
  - Teilprobleme der Größe n/b (für Konstante $b > 1$)
  - nicht-rekursiver "conquer" Aufwand $f(n)$ (für Funktion $f : \mathbb{R} \to \mathbb{R}$)
  - Basisfall-Aufwand d (some constant $d > 0$)

Die Master-Methode

$$T(n) = \begin{cases} 0 & n \leq 1 \\ 2 \cdot T(n/2) + n & n \geq 2 \end{cases}$$

- Allgemeines D&C-Verfahren
  - a rekursive Aufrufe (für Konstante $a > 0$)
  - Teilprobleme der Größe n/b (für Konstante $b > 1$)
  - nicht-rekursiver "conquer" Aufwand $f(n)$ (für Funktion $f : \mathbb{R} \to \mathbb{R}$)
  - Basisfall-Aufwand d (some constant $d > 0$)

$\rightsquigarrow$ Laufzeit $T(n)$ erfüllt Rekursion

$$T(n) = \begin{cases} a \cdot T\left(\frac{n}{b}\right) + f(n) & n > 1 \\ d & n \leq 1 \end{cases}$$

$(n = b^k, k \in \mathbb{N}_0)$

Die Master-Methode

- Allgemeines D&C-Verfahren
  - a rekursive Aufrufe (für Konstante $a > 0$)
  - Teilprobleme der Größe n/b (für Konstante $b > 1$)
  - nicht-rekursiver "conquer" Aufwand $f(n)$ (für Funktion $f : \mathbb{R} \to \mathbb{R}$)
  - Basisfall-Aufwand d (some constant $d > 0$)

$\rightsquigarrow$ Laufzeit $T(n)$ erfüllt Rekursion

$$\boxed{T(n) = \begin{cases} a \cdot T\left(\frac{n}{b}\right) + f(n) & n > 1 \\ d & n \leq 1 \end{cases}} \qquad (n = b^k, k \in \mathbb{N}_0)$$

Theorem 3.4 (Master Theorem)

Mit $c := \log_b(a)$, gilt für die obige Rekursion:

(a) $T(n) = \Theta(n^c)$ falls $f(n) = O(n^{c-\varepsilon})$ für konstantes $\varepsilon > 0$.

(b) $T(n) = \Theta(n^c \log n)$ falls $f(n) = \Theta(n^c)$.

(c) $T(n) = \Theta(f(n))$ falls $f(n) = \Omega(n^{c+\varepsilon})$ für konstantes $\varepsilon > 0$ **und** f erfüllt die *Regularitätsbedingung* $\exists n_0, \alpha < 1\ \forall n \geq n_0 : a \cdot f\left(\frac{n}{b}\right) \leq \alpha f(n)$.

Regularitätsbedingung

Fall (c) braucht leider die technische Regularitätsbedingung.

Aber, für einfache Funktionen f gilt diese immer.

Lemma 3.5 (Regularitätsbedingung)

Für $f(n) = n^\beta \lg^\gamma(n)$ mit Konstanten $\beta > \log_b(a)$ und γ gilt die Regularitätsbedingung. ◀